■# IBI Bank — Database Normalization Document

1. Introduction

Normalization organizes relational data to reduce unnecessary duplication and prevent insertion, update, and deletion anomalies. This document explains the intended normalization of the IBI Bank database. The final claims should be checked against the actual table definitions and functional dependencies.

2. First Normal Form (1NF)

A relation is in First Normal Form when each field stores an atomic value and repeating groups are avoided.

Design examples:

• Customer contact details are stored as individual fields in the customer table.

• Account details are represented as account records rather than a list of accounts inside one customer field.

• Each transaction is recorded as an individual row in the transaction table.

• Each EMI payment is represented by a separate row in emi_payment.

3. Second Normal Form (2NF)

A relation is in Second Normal Form when it is in 1NF and every non-key attribute depends on the whole candidate key, rather than only part of a composite key.

Design examples:

• Tables use their own record identifiers, such as account_id, transaction_id, loan_id, and emi_id.

• Account attributes such as balance and status belong to the account record rather than being repeated in every transaction record.

• EMI-specific details are stored in emi_payment, while the related loan is identified by loan_id.

Where a table uses a composite key, inspect its non-key attributes to confirm that no partial dependency exists.

4. Third Normal Form (3NF)

A relation is in Third Normal Form when it is in 2NF and non-key attributes do not depend transitively on a key through another non-key attribute.

Design examples:

• Branch details are maintained in branch rather than repeated as full branch information in every account.

• Account category information is maintained in account_type and referenced by accounts through account_type_id.

• Standard loan category/rate information is represented in loan_type, separately from individual loan records.

• Loan payment details are stored separately from the loan's principal, rate, and tenure.

Important implementation note: The loan table currently has a loan_type field, while loan_type also stores category information. Confirm whether the loan uses loan_type_id or a text category and whether the design intentionally duplicates standard rate information. If category and rate are repeated, explain the business reason or consider a foreign-key relationship to loan_type in a future schema revision. Do not alter the working database solely to match this document.

5. Reducing update anomalies

• Keeping branch data separate helps avoid updating the same branch details across many account rows.

• Keeping account-type data separate helps centralize account-type rules.

• Keeping transactions separate from account balances preserves transaction history while the current balance is stored on the account.

• Keeping loan and EMI records separate allows multiple scheduled payments to be recorded for one loan.

• Keeping login identities separate from customer profiles separates authentication concerns from banking profile information.

6. Integrity and security

• Primary keys uniquely identify records.

• Foreign keys, where declared, protect relationships between records.

• Unique constraints should protect values that must not be duplicated, such as account numbers and login emails, according to the implemented schema.

• Amount validation and application-level checks help reject invalid transactions.

• The account balance audit trigger supports traceability of balance changes.

• The transaction amount validation trigger rejects invalid transaction amounts, according to the implemented trigger design.

• Passwords and verification codes must not be stored in project documentation as real values.

7. Conclusion

The schema separates core banking entities such as customers, accounts, branches, account types, transactions, transfers, loans, and EMI payments into related tables. This separation supports data consistency and maintainability. Final normalization claims should be validated against the actual schema, functional dependencies, and constraints exported from MySQL.